PART A

Candidate Brief

We care about engineering judgment, not typing speed. You are encouraged to use AI tools (Claude, Copilot, ChatGPT, whatever you like) for this exercise — the same way you would on the job. We're not testing whether you can produce code without help; we're testing whether you can direct a problem to a good outcome and reason about the parts that matter.

If you move forward to the next stage, you'll be asked to present your solution and discuss your approach during the interview. The conversation will focus on your reasoning, design decisions, trade-offs, and what you learned throughout the process—not simply on the code itself.

The problem

Build a webhook delivery service: a system that accepts events from internal producers and reliably delivers them to customer-registered HTTP endpoints.

Concretely, the service should:

- Accept events over (e.g. `POST /events` with a customer/subscription identifier and a JSON payload).
- Deliver each event to the customer's registered endpoint via an HTTP **POST** .
- Retry delivery when the customer's endpoint is unavailable or returns an error.
- Give some way to see the delivery status of an event (delivered / pending / failed).

A registered endpoint is just a URL the customer gave us. It can be slow, return errors, or be completely down for extended periods — and we still shouldn't lose their events.

Decisions we're deliberately leaving to you

This is the part we care about most. The requirements above are intentionally underspecified. We want to see how you reason about the following — there are no single right answers, but there are answers you should be able to defend:

- Delivery semantics. What guarantee do you offer the customer — at-most-once, at-least-once, exactly-once? What does that guarantee cost, and what does it demand of the customer?
- Retry behavior. How many times, how far apart, and what's the terminal state when an endpoint just never comes back?
- Long outages. A customer's endpoint is down for six hours. What does your service do during that window, and what happens to everyone else's deliveries meanwhile?
- Ordering. Do events for a given customer arrive in the order they were produced? Should they?

If you advance to the next stage, you'll present your solution and discuss these design choices and trade-offs during the interview.

Out of scope (don't build these)

To keep this to a few hours: no auth/login system, no UI, no multi-tenant billing, no production deployment. A single-process service with a real datastore (or a well-justified in-memory store) is fine. Mocking the customer endpoints is expected.

Time budget

Please cap yourself at ~3 hours. We mean it — we'd rather see a smaller, well-reasoned solution with clear notes on what you'd do next than something exhaustive. If you hit the cap mid-thought, stop and write down what you'd have done.

What to submit

1. The code, in a repo or zip, with a one-line run command.
2. A short DECISIONS.md (half a page is plenty) covering the four decisions above and anything you deliberately left out.
3. Please include the complete transcript (or exported conversation) of any AI assistant(s) you used while completing this assignment. Do not edit, summarize, or remove parts of the conversation. We're interested in understanding your workflow, not judging the prompts themselves.
4. Optional : You may also include a brief reflection on your AI-assisted workflow and decision-making process while completing the assignment. (e.g. learnings, prompting strategy, useful prompts, or challenges encountered).

How we'll evaluate

We'll evaluate the quality of your engineering decisions, how you handle trade-offs and failure scenarios, and how well you communicate your reasoning.

If you advance to the next stage, the interview will include:

- A walkthrough of your solution and the decisions behind it.
- A live extension of the project, where you'll build on your

implementation (AI tools are welcome).

- A collaborative review of a small portion of your code, discussing possible improvements and trade-offs.

We're explicitly not evaluating your choice of framework, how little code you write, or achieving a particular test coverage percentage.